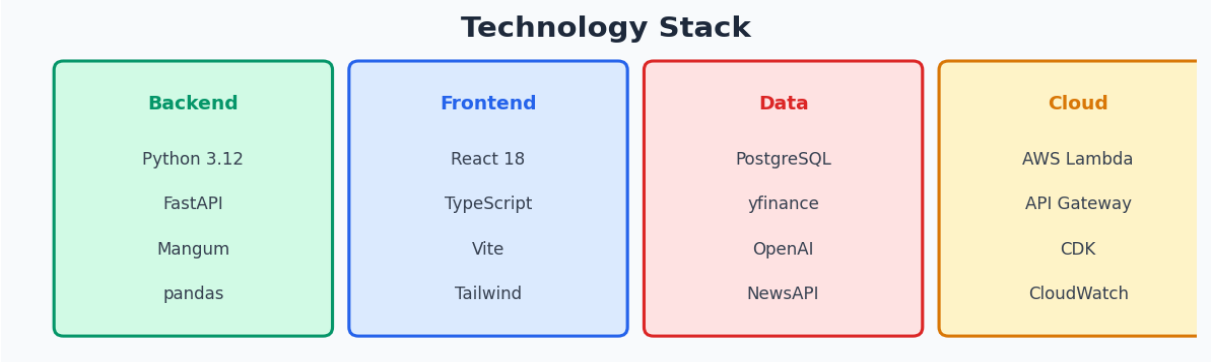
Stock Monitoring & Analysis System

A serverless, AI-powered stock analysis platform on AWS



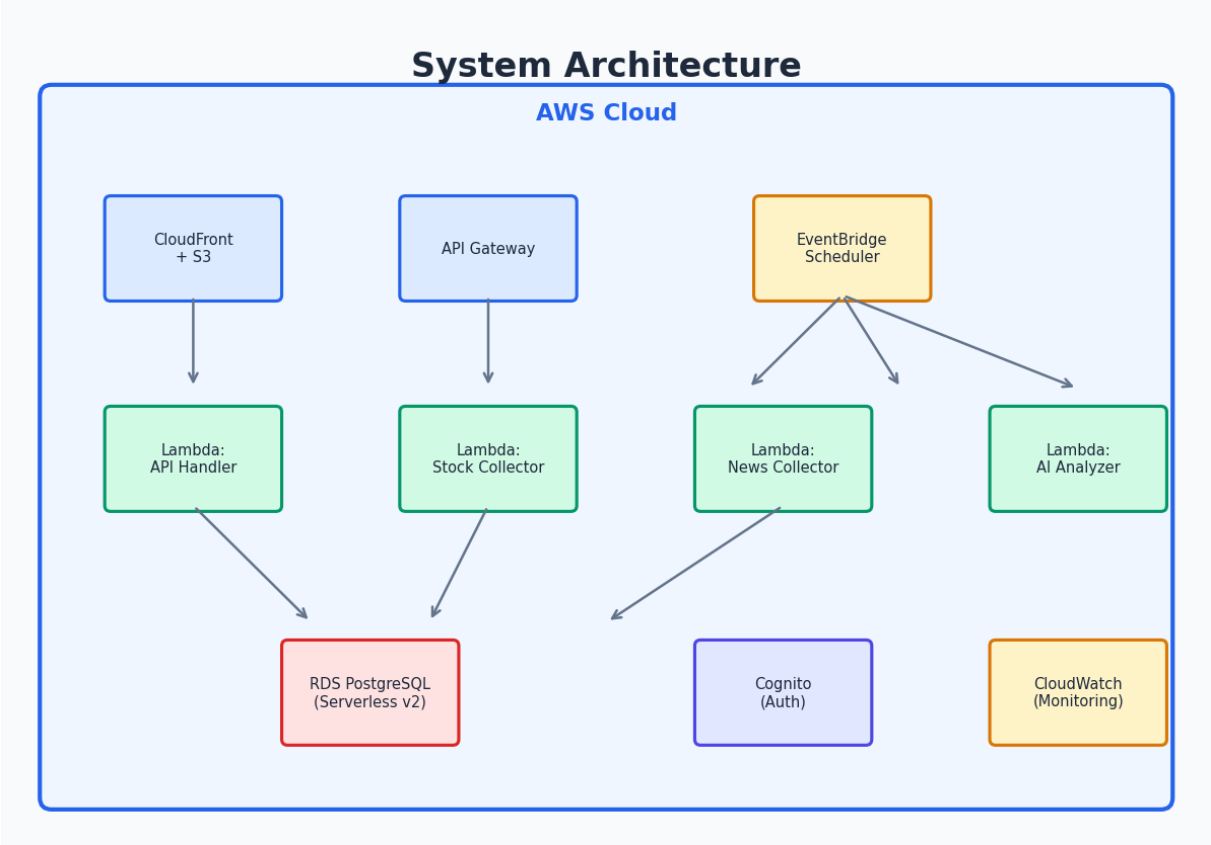
Designed for 10-100 users • Minimal hosting cost • Daily AI recommendations

System Overview

The Stock Monitoring and Analysis System is a cloud-hosted web application that collects daily stock market data and news, applies AI-driven analysis to generate buy/hold/sell recommendations, and provides personalized portfolio suggestions through a web dashboard.

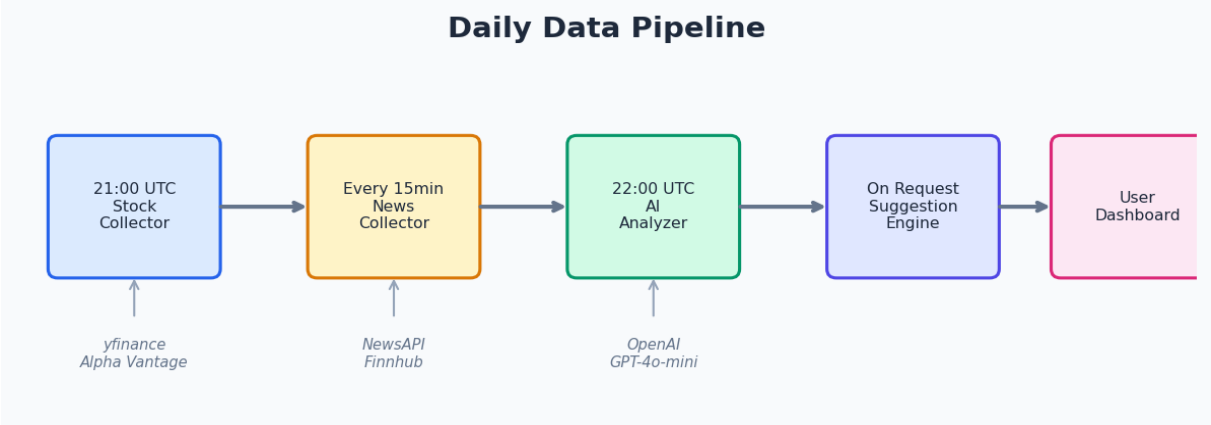
- **1000+ stocks monitored** — categorized by sector and company size
- **Daily AI analysis** — GPT-4o-mini generates BUY/HOLD/SELL recommendations
- **Encrypted portfolios** — AES-256-GCM via AWS KMS
- **Personalized suggestions** — filtered by sector, size, and risk preferences
- **Serverless architecture** — scales to zero, minimal cost at low usage
- **Automated pipeline** — EventBridge schedules for daily data collection

Architecture



Daily Data Pipeline

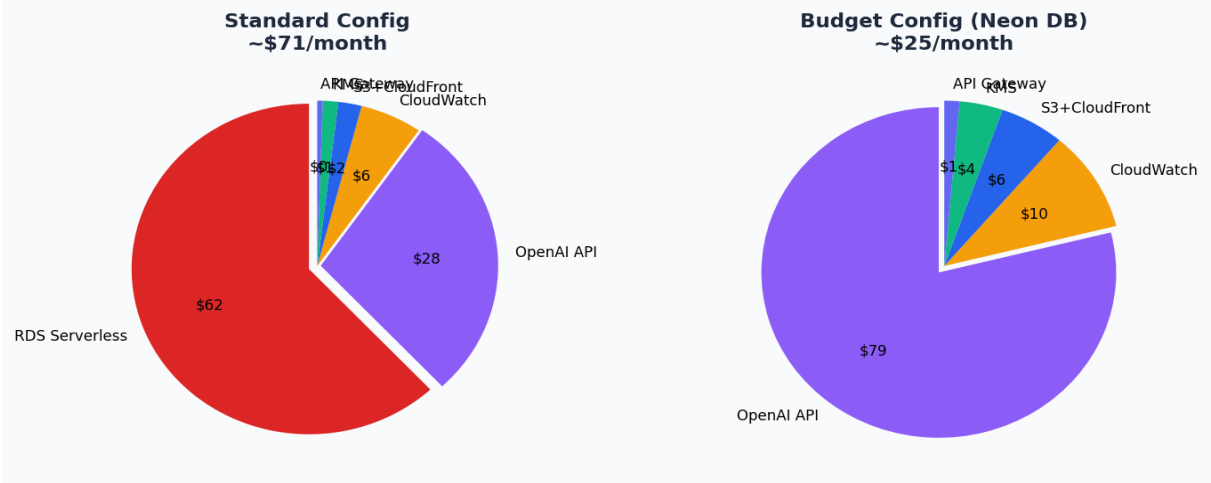
The system operates on a daily schedule, collecting data after market close and generating AI analysis within hours. News is collected continuously every 15 minutes for real-time market sentiment.



Component	Schedule	Description
Stock Collector	21:00 UTC daily	Fetches OHLCV data for 1000+ tickers via yfinance
News Collector	Every 15 minutes	Polls NewsAPI + Finnhub, generates AI summaries
AI Analyzer	22:00 UTC daily	Calculates SMA/RSI/MACD, calls GPT-4o-mini
Suggestion Engine	On user request	Compares portfolio vs analysis, filters by prefs

Cost Estimation

Two deployment configurations are available depending on budget constraints. The budget configuration uses Neon's free-tier PostgreSQL to eliminate the largest cost (RDS), bringing total monthly spend under $30.



Standard Configuration

Service	Usage	Monthly Cost
Lambda	~50K invocations/month	$0 (free tier)
API Gateway	~100K requests/month	$0.35
RDS Serverless v2	0.5 ACU minimum	$43.80
S3 + CloudFront	Frontend hosting	$1–2
Cognito	<50K users	$0 (free tier)
CloudWatch	Logs + metrics + alarms	$3–5
OpenAI API (GPT-4o-mini)	~1000 stocks × 30 days	$15–25
KMS	1 key + decrypt calls	$1
TOTAL		**$65–77/month**

Budget Configuration (≤$30/month)

Service	Monthly Cost
Lambda + API Gateway	$0.35
Neon PostgreSQL (free tier)	$0
S3 + CloudFront	$1–2

Cognito	$0
CloudWatch (basic)	$2–3
OpenAI API (GPT-4o-mini)	$15–25
KMS	$1
TOTAL	**$20–30/month**

Security & Design Decisions

• **Encryption at rest** — RDS storage encryption, S3 bucket encryption, portfolio data encrypted with AES-256-GCM via AWS KMS

• **Encryption in transit** — All traffic over HTTPS (CloudFront + API Gateway)

• **Authentication** — AWS Cognito with JWT tokens, password policy (8+ chars, uppercase, lowercase, digit), account lockout after 5 failed attempts

• **Authorization** — Cognito authorizer on API Gateway, user-scoped portfolio access

• **Least privilege** — Separate IAM roles per Lambda with minimal permissions

• **Secrets management** — API keys via AWS Secrets Manager, never in code

• **Session security** — 30-minute inactivity timeout, token expiry enforcement

Database Schema

PostgreSQL with 7 tables designed for efficient time-series queries and encrypted portfolio storage:

Table	Purpose	Key Fields
stocks	Watchlist (1000+ tickers)	ticker PK, sector, company_size
stock_data	Daily OHLCV prices	ticker, trading_date, OHLCV, UNIQUE
news_summaries	AI-summarized articles	title, tickers[], summary (≤500)
analysis_results	Daily recommendations	BUY/HOLD/SELL, risk, confidence
users	Cognito user mirror	UUID id, email
portfolios	Encrypted holdings	encrypted_data TEXT (AES-256)
user_preferences	Filter settings	sectors[], sizes[], max_risk

API Endpoints

Method	Path	Description
`POST`	`/api/auth/register`	Register new user
`POST`	`/api/auth/login`	Login, returns JWT
`GET`	`/api/portfolio`	Decrypted user portfolio
`PUT`	`/api/portfolio/stocks`	Add stock to portfolio
`DELETE`	`/api/portfolio/stocks/{ticker}`	Remove stock
`GET`	`/api/suggestions`	Personalized BUY/SELL suggestions

GET	/api/stocks	List monitored stocks (filtered)
GET	/api/stocks/{ticker}/analysis	Latest analysis for ticker
GET/PUT	/api/preferences	User filter preferences
GET	/api/health	System health check